HACKERS HRD · 특강

AI X n8n 업무자동화 실전 프로젝트

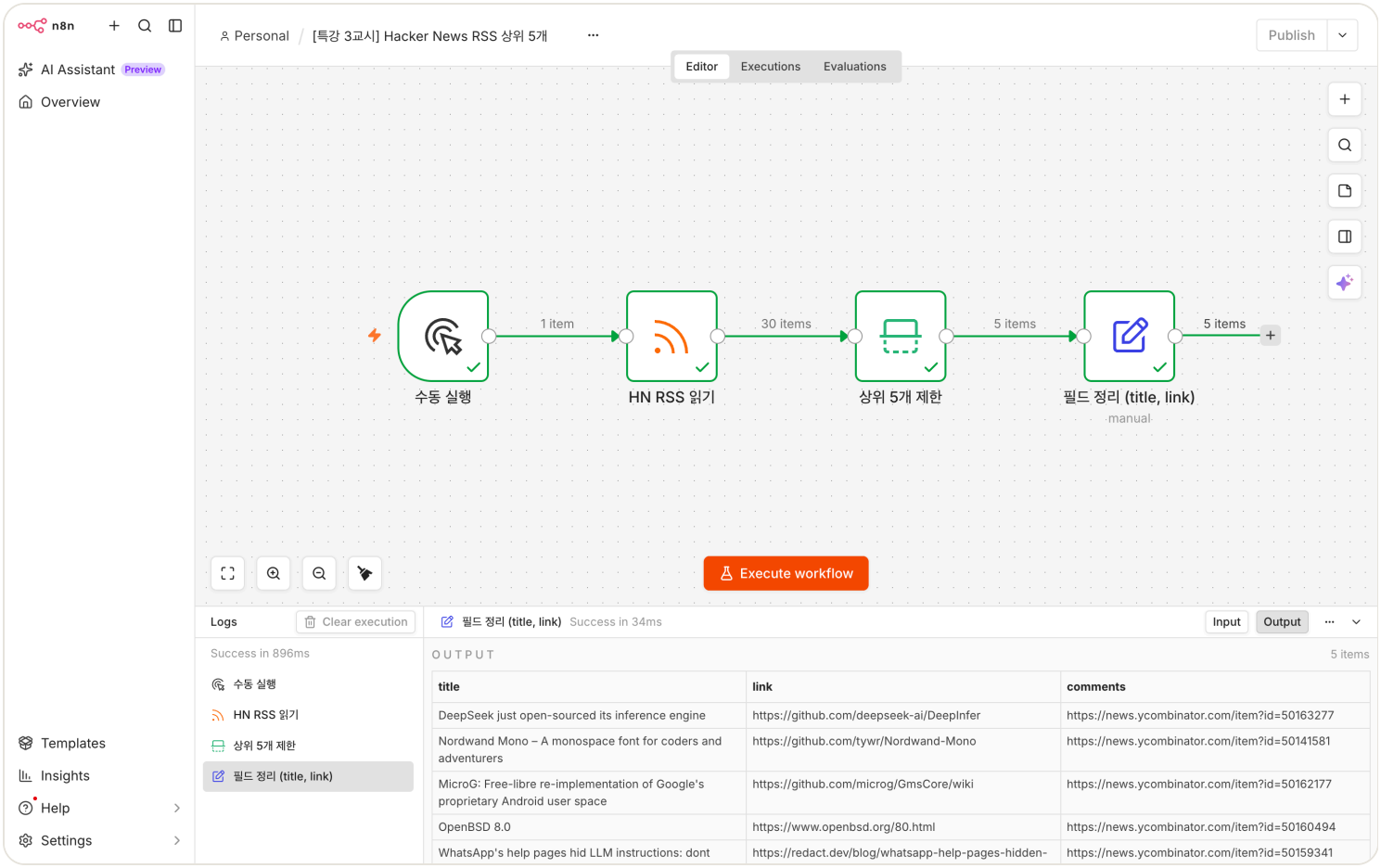
3시간 특강 · 중소조선연구원 · 2026.10.26 (월) 13:00–16:00

오프닝 · n8n 과 AI, 그리고 워크플로우

n8n 이란?

> **n8n = 업무를 “순서도”로 그려두면, 그 순서대로 24시간 대신 일해 주는 자동화 도구**

- 이름 뜻: **n**odemation → n + 8글자 + n (노드로 하는 자동화)
- 화면에서 **블록(노드)을 끌어다 놓고 선으로 연결**하면 끝 — 코딩을 몰라도 시작할 수 있습니다.
- **수백 가지 서비스와 연결**: 메일 · 구글시트 · 슬랙/텔레그램 · 노션 · DB · 웹사이트 · **AI(ChatGPT, Claude, Gemini)** …
- **내 컴퓨터/회사 서버에 설치 가능**(셀프호스팅) → 사내 데이터를 밖으로 보내지 않고 자동화
- 비슷한 도구: Zapier, Make — n8n 은 **자유도 · 가격 · AI 기능**이 강점



n8n 화면 — 블록(노드)을 선으로 이어 만든 워크플로우

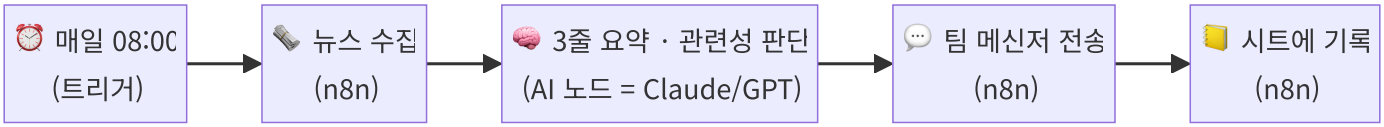
Claude · Codex 와 무엇이 다른가요?

요즘 Claude, ChatGPT, Codex 같은 AI 가 “알아서 다 해준다”는데, n8n 은 왜 필요할까요?

	🧠 Claude · ChatGPT	👩‍💻 Claude Code · Codex	🏭 n8n
한마디로	**똑똑한 상담 직원**	**AI 개발자**	**공장 생산 라인 / 배관**
일하는 방식	내가 물어보면 대답	내가 시키면 코드를 짜고 고침	**정해진 순서**대로 반복 실행
언제 일하나	내가 말을 걸 때만	내가 시킬 때만	**조건이 되면 스스로** (매일 9시, 메일 도착, 버튼 클릭 …)
결과	매번 조금씩 다를 수 있음	매번 다를 수 있음	**항상 같은 순서 · 같은 규칙**
기록	대화창	코드 저장소	**실행마다 단계별 기록** (Executions)
잘하는 일	생각 · 요약 · 글쓰기 · 판단	프로그램 · 웹페이지 만들기	**연결 · 반복 · 정시 실행 · 전달**

💡 경쟁 관계가 아니라 팀입니다.

- n8n 라인 중간에 **AI 노드(Claude/GPT)를 "두뇌 블록"으로 끼워** 요약·분류·판단을 맡기고,
- **Claude Code · Codex · Cursor 에게** 웹페이지나 n8n 워크플로우 초안을 **만들게** 하고,
- 다 만든 n8n 워크플로우는 반대로 **AI 가 부르는 도구(MCP)** 가 되기도 합니다.



정리: AI 는 **“생각”**, n8n 은 **“언제 · 어떤 순서로 · 어디에 연결할지”** 를 맡습니다.

워크플로우를 만든다는 것

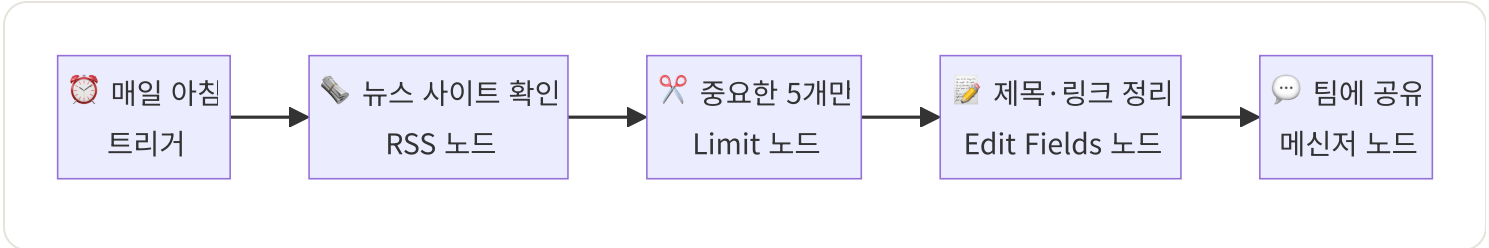
> **워크플로우 = 사람이 하던 일을 “단계”로 쪼개서, 블록으로 이어 붙인 순서도**

요리 레시피와 똑같습니다. 🍳 **“언제(트리거) → 무엇을(노드) → 어떤 순서로(연결선)”**

레시피	워크플로우	n8n 에서는
“저녁 6시가 되면”	**트리거** — 언제 시작?	Schedule · Webhook · 수동 실행
“재료 손질 → 볶기 → 담기”	**노드** — 각 단계의 작업	RSS 읽기 · AI 요약 · 메일 보내기 …
화살표 순서	**연결선** — 결과가 다음 단계로 흘러감	노드 사이의 선
완성된 요리	**결과** — 어디로 보낼까?	메신저 · 시트 · 메일 · 웹페이지

연습: 내 업무를 쪼개 보기

“매일 아침 업계 뉴스를 확인해서 중요한 것만 팀에 공유한다” 👉



- 사람이 머릿속으로 하던 일을 **눈에 보이는 순서도**로 꺼내는 것 → 이것이 “워크플로우를 만든다”는 뜻

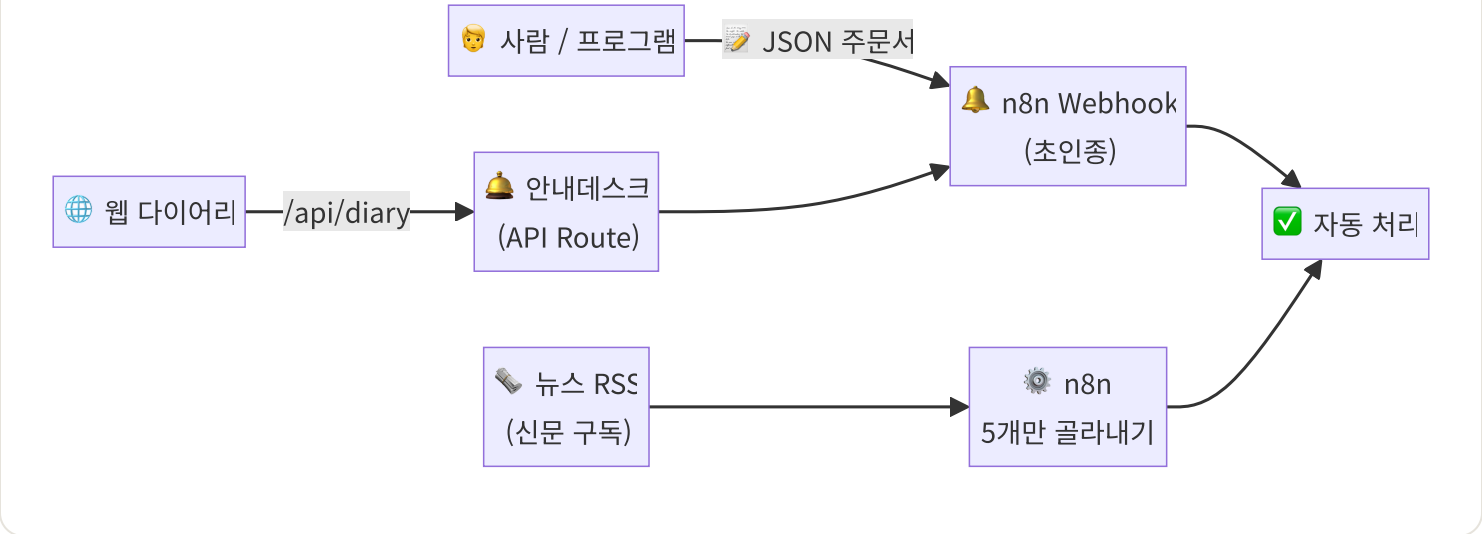
- 한 번 만들면 **매일, 정확하게, 대신** 실행됩니다. 오늘 3교시에 바로 이것(앞의 4단계)을 만듭니다.

오늘의 목표

> **코드를 거의 치지 않고**, 업무 자동화의 “배관” 3개를 끝까지 연결해 봅니다.

자동화는 결국 이 한 문장입니다.

누가 → 무엇을 → 어디로 보내는가



시간	교시	오늘 만들 것	과제
13:00	오프닝	n8n 이란? · AI 와의 차이 · 워크플로우란?	
13:20	**1교시**	n8n이 JSON을 받는 “초인종”	문제 1 (30점)
14:10	**2교시**	웹 다이어리 → 안내데스크 → n8n, 인터넷에 배포	문제 2 (40점)
15:05	**3교시**	최신 뉴스 5개만 골라오기	문제 3 (30점)
15:40	확장 · Q&A	우리 연구원 업무에 붙여보기 · 과제 제출	

오늘 꼭 가져갈 단어 8개

단어	한 줄 비유
API	🍽️ 식당 **웨이터** — 메뉴판(규칙)대로 주문을 주방에 전달
JSON	📝 정해진 칸이 있는 **주문서**
Webhook	🔔 n8n에 달린 **초인종**
API Route	🔔 회사 **안내데스크**
환경변수	🔐 비밀값을 넣어두는 **금고**
배포	🏪 내 식당을 **번화가에 개업**
RSS	🗞️ 신문 **정기구독**
아이템(item)	📦 **컨베이어 벨트 위 상자**

준비물 체크

- [ ] **n8n** 로그인 (n8n Cloud 체험판 또는 설치형)
- [ ] **hoppscotch.io** 접속 (설치 없음 · 구글 로그인하면 요청 저장) — Cursor + REST Client 도 OK
- [ ] **GitHub** 계정 · **Vercel** 계정 (Continue with GitHub)

> 🙋 막히면 **손 들기** · 스트리밍은 **채팅에** 🙋

📦 **셀프 학습 키트 · 집에서 다시 하기**

- 🎁 **키트 첫 화면** — 매뉴얼 · 워크플로우 · 그림을 한곳에
- 📘 **셀프 실습 매뉴얼** 1장 — n8n Cloud 가입 / Docker 로 내 PC 설치 · Cursor · REST Client

1교시 · n8n Webhook과 REST Client

📚 이번 교시 필수 개념

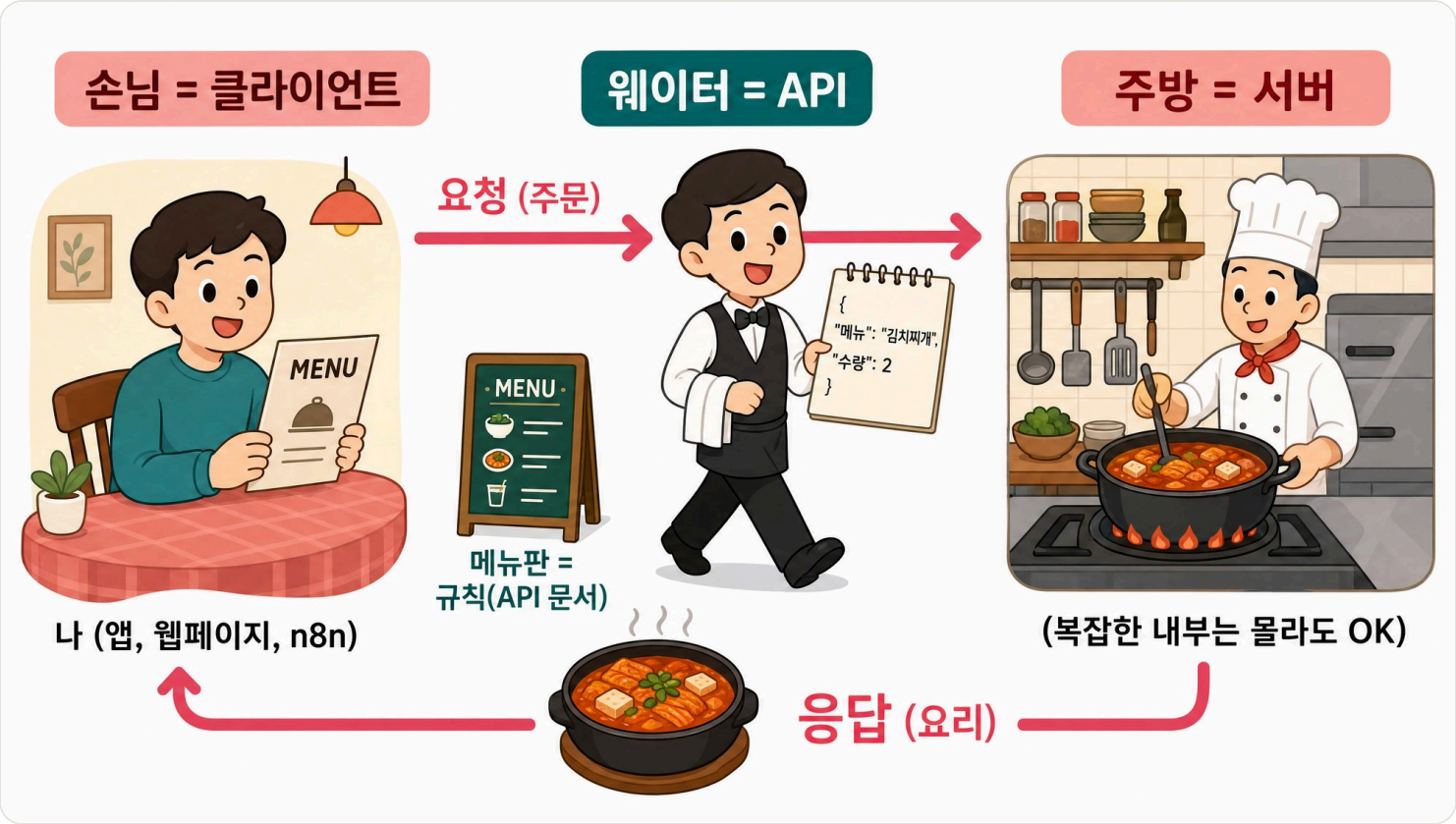
🍽️ API · 📮 URL(주소) · ✉️ GET · POST · 🧾 상태코드 · 📝 JSON

🔔 Webhook · 🎭 Test · Production URL · 🚢 Hoppscotch

🗂️ 워크스페이스 · 컬렉션 · 🔀 CORS · Proxy

목표: **“다른 프로그램이 n8n 에게 JSON 주문서를 보내면, n8n 이 받는다”** 를 눈으로 확인

API — 식당 웨이터



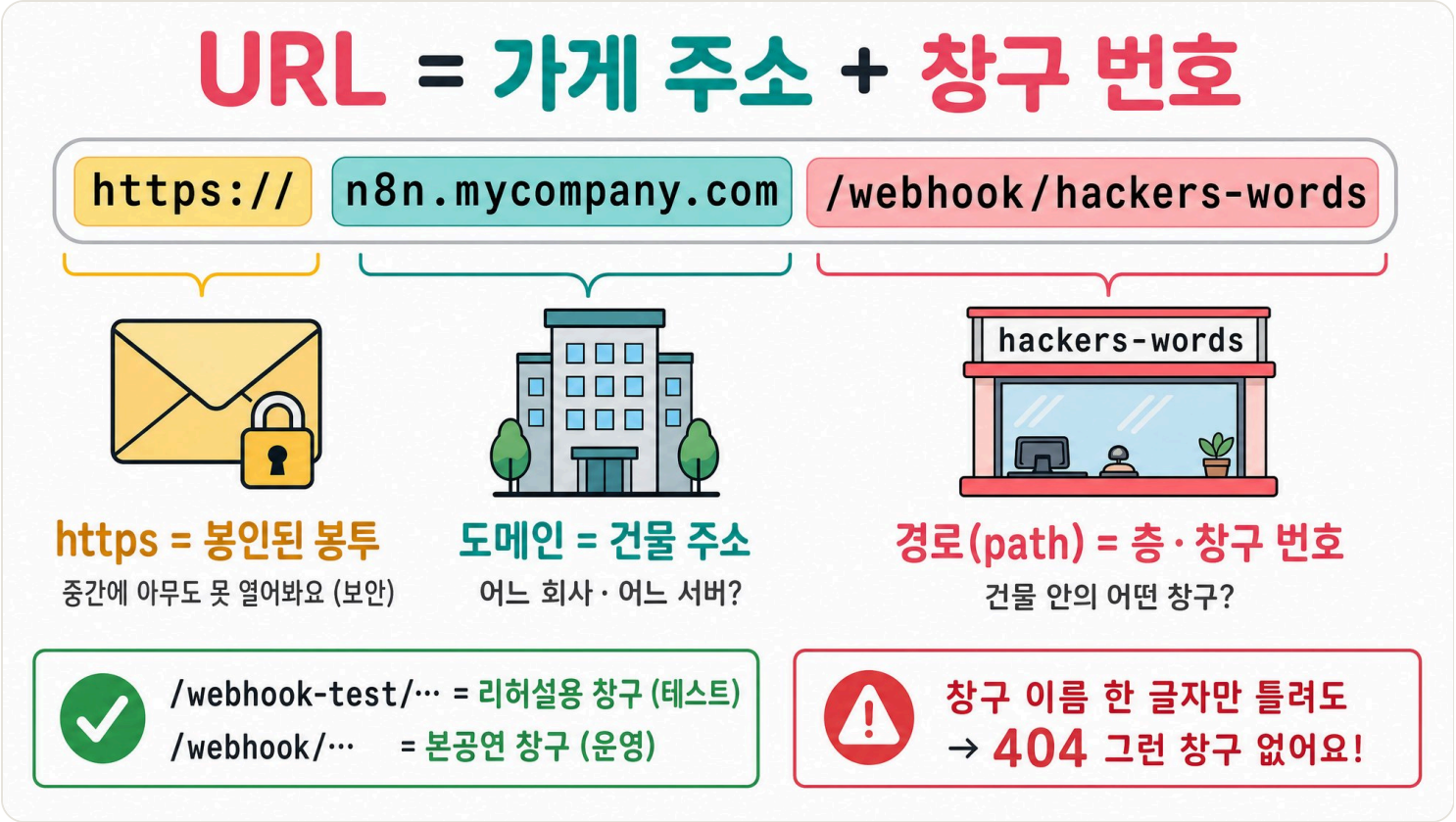
API = 식당 웨이터 — 손님(클라이언트)의 주문을 주방(서버)에 전달하고 요리(응답)를 가져온다

- 손님이 **주방에 들어가 요리법을 배울 필요가 없습니다.**
- **메뉴판(규칙)** 을 보고 **웨이터에게 주문**만 하면 요리(결과)가 나옵니다.
- **API = 프로그램끼리 대화하는 규칙**
  - ChatGPT, 카카오톡, 지도… 그리고 **n8n의 거의 모든 노드가 API로 움직입니다.**

> 💬 우리 연구원에서 “웨이터” 역할을 하는 사람은 누구일까요?

URL — 가게 주소와 테이블 번호 📮

API 를 부르려면 **어디로 주문할지 주소**가 필요합니다.

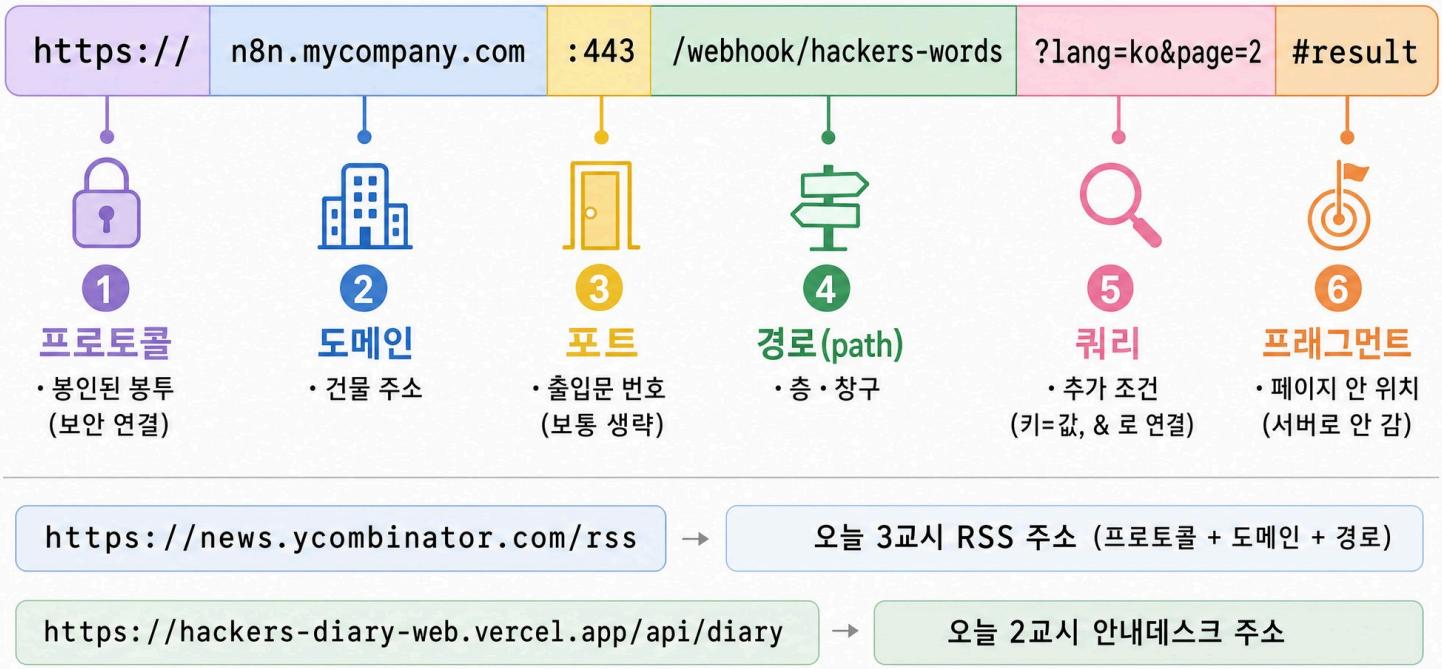


URL = 가게 주소 + 창구 번호

- **https** = 봉인된 봉투 (중간에 남이 못 열어봄)
- **도메인** = 건물 주소 — 어느 회사·서버인지
- **경로(path)** = 층·호수 — 그 안의 **어떤 창구**인지. 오늘 만들 창구 이름이 `hackers-words`
- 경로가 한 글자라도 틀리면 → **404 "그런 창구 없는데요?"**

주소 한 줄 완전 해부 (Full URL 예시)

주소(URL) 한 줄 완전 해부



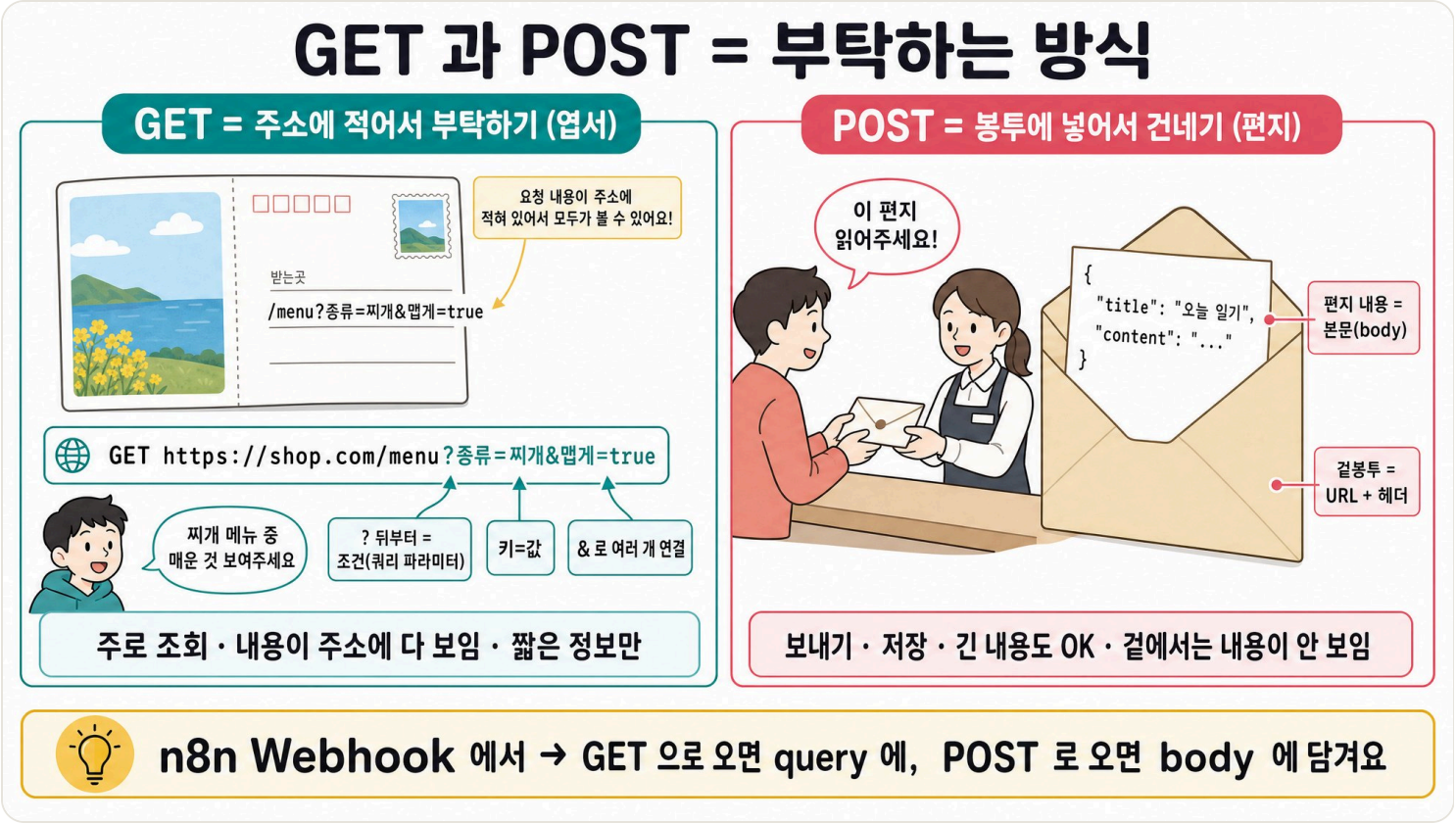
주소(URL) 한 줄 완전 해부 — 프로토콜 · 도메인 · 포트 · 경로 · 쿼리 · 프래그먼트

```
https://n8n.mycompany.com:443/webhook/hackers-words?lang=ko&page=2#result
└─①─┘ └──────②──────┘└③┘└──────④──────┘└──────⑤──────┘└─⑥─┘
```

번호	부분	예시	비유 · 설명
①	프로토콜	`https://`	봉인된 봉투 — 보안 연결
②	도메인	`n8n.mycompany.com`	건물 주소 — 어느 서버?
③	포트	`:443`	출입문 번호 — https 는 443 이 기본이라 **보통 생략**
④	경로	`/webhook/hackers-words`	층 · 창구 — 서버 안의 어느 기능?
⑤	쿼리	`?lang=ko&page=2`	추가 조건 — `?` 로 시작, `키=값` , 여러 개는 `&`
⑥	프래그먼트	`#result`	페이지 안의 위치(책갈피) — **서버로는 안 보내짐**

> 💡 매일 쓰는 주소에도 다 들어 있습니다. 네이버 검색 결과 주소 `https://search.naver.com/search.naver?query=n8n` 의 `?query=n8n` 이 바로 **쿼리** 예요.

주문 방식(메서드) — 엽서 vs 편지 ✉️



GET 과 POST = 부탁하는 방식 — 주소에 적는 엽서 vs 봉투에 넣는 편지

메서드	비유	내용은 어디에?	주로
GET	📮 **엽서** — 주소란에 부탁을 **적어서** 보냄	**URL 쿼리** `?종류=찌개&맵게=true` (누구나 보임)	조회 · 검색
POST	✉️ **편지** — "이 편지 읽어주세요!" **봉투 안에** 넣어 건넴	**본문(body)** 에 JSON (겉에서 안 보임)	보내기 · 저장 ⭐ 오늘의 주인공
PUT / DELETE	주문 변경 / 취소	본문 / URL	수정 / 삭제

같은 부탁을 두 방식으로 써 보면 👇

```
### GET — 부탁 내용이 주소에 그대로 보임
GET https://n8n.mycompany.com/webhook/hackers-words?word=고양이&word2=커피

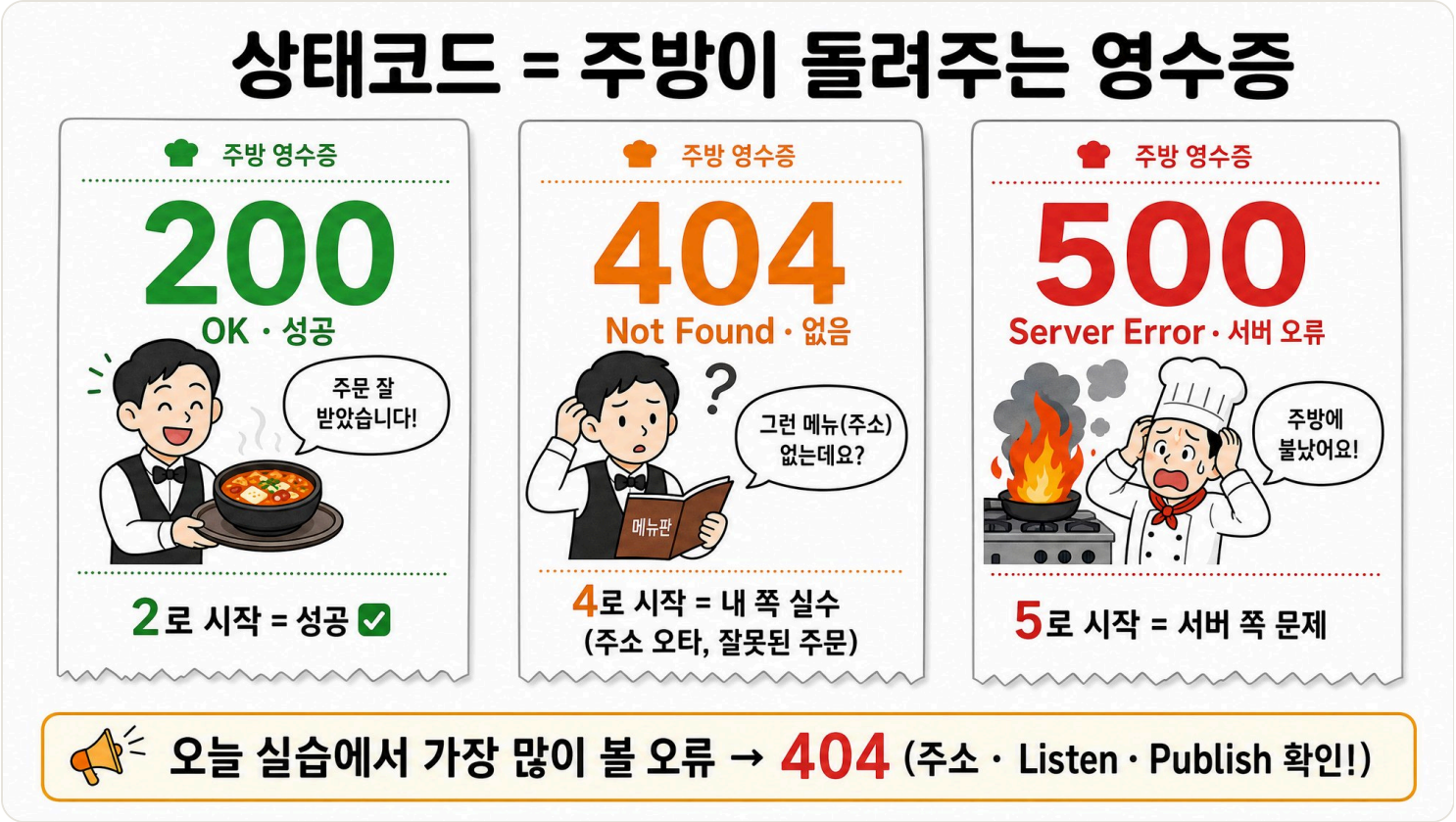
### POST — 주소는 창구까지만, 부탁 내용은 봉투(body) 안에
POST https://n8n.mycompany.com/webhook/hackers-words
Content-Type: application/json

{ "words": ["고양이", "커피", "회의"] }
```

⚠️ 위 GET 줄은 **비교용 예시**입니다. 실습 웹훅은 POST 로 만들었기 때문에 GET 으로 부르면 404 가 납니다 (HTTP Method 를 GET 으로 바꾸면 동작).

🔎 **n8n Webhook 에서 확인하는 법**: GET 으로 오면 OUTPUT 의 `query` 칸에, POST 로 오면 `body` 칸에 담깁니다. 1교시 정답 화면에서 `query` 는 비어 있고 `body › words` 에 값이 있는 이유가 바로 이것!

영수증(상태코드) 🧾

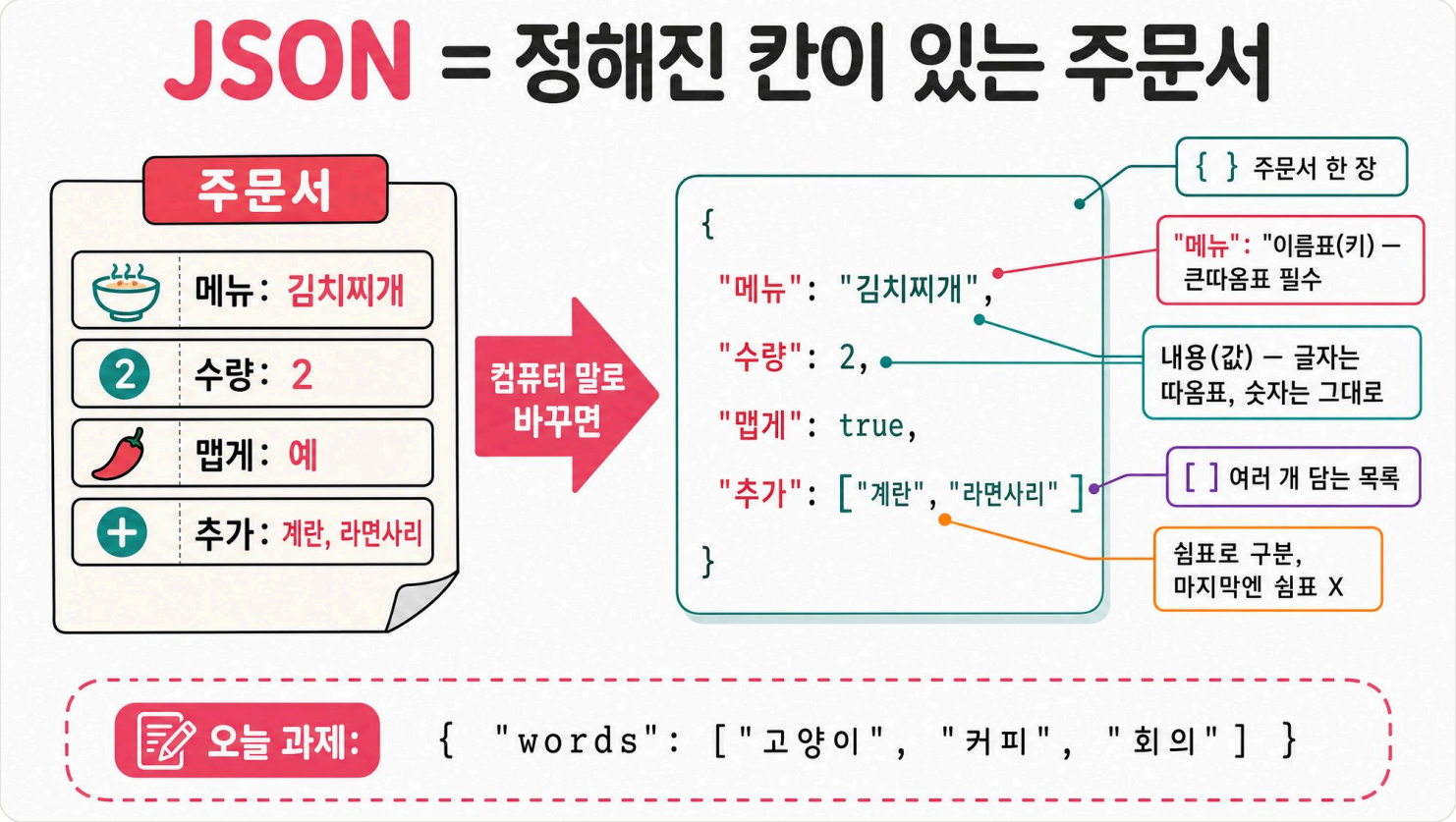


상태코드 = 주방이 돌려주는 영수증 — 200 · 404 · 500

상태코드	식당에서는	의미
200 (2xx)	“주문 잘 받았습니다”	✅ 성공
404 (4xx)	“그런 메뉴(주소) 없는데요?”	❌ **내 쪽 실수** — 주소 오타 · Listen 안 누름 · Publish 안 함 → **오늘 제일 많이 볼 오류**
500 (5xx)	“주방에 불났어요”	🔥 **서버 쪽 문제** — 잠시 후 다시 / 관리자에게

> 첫 숫자만 보면 됩니다: **2 = 성공, 4 = 내가 고칠 것, 5 = 서버가 고칠 것**

JSON — 주문서 양식



JSON = 정해진 칸이 있는 주문서

정해진 칸이 있는 주문서처럼 **이름표(키) : 내용(값)** 으로 적습니다.

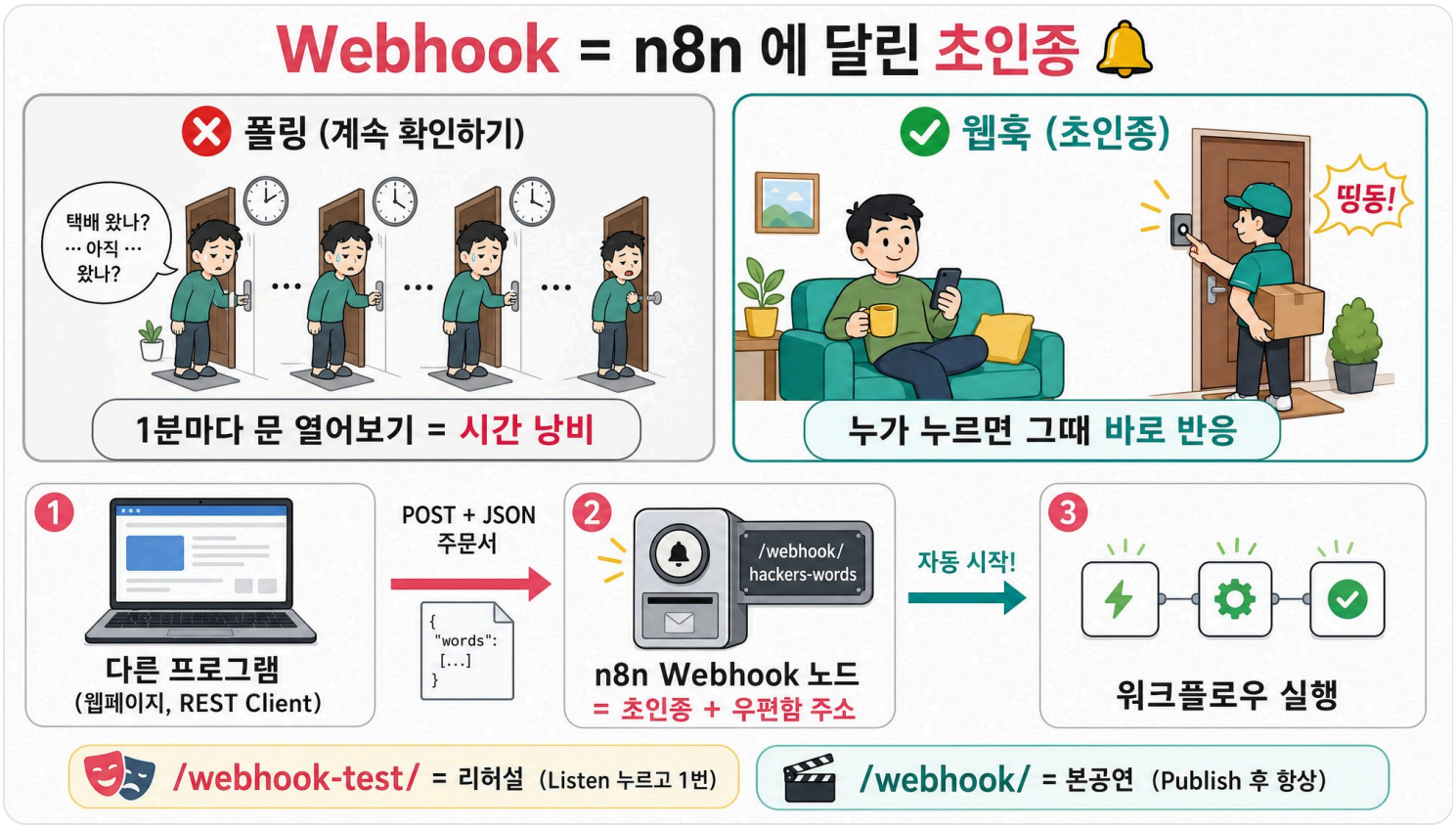
```
{
  "메뉴": "김치찌개",
  "수량": 2,
  "맵게": true,
  "추가": ["계란", "라면사리"]
}
```

규칙	설명
`{ }`	주문서 한 장
`"키": 값`	이름표는 **반드시 큰따옴표**
`[ ]`	여러 개를 담는 **목록(배열)**
`,`	항목 구분 — **마지막 항목 뒤에는 쉼표 금지!**

오늘 과제 1의 주문서 👇 “words 라는 이름표에 단어 **3개짜리 목록**”

```
{ "words": ["고양이", "커피", "회의"] }
```

Webhook — 초인종



Webhook = n8n 에 달린 초인종 — 폴링 vs 웹훅, 그리고 동작 순서

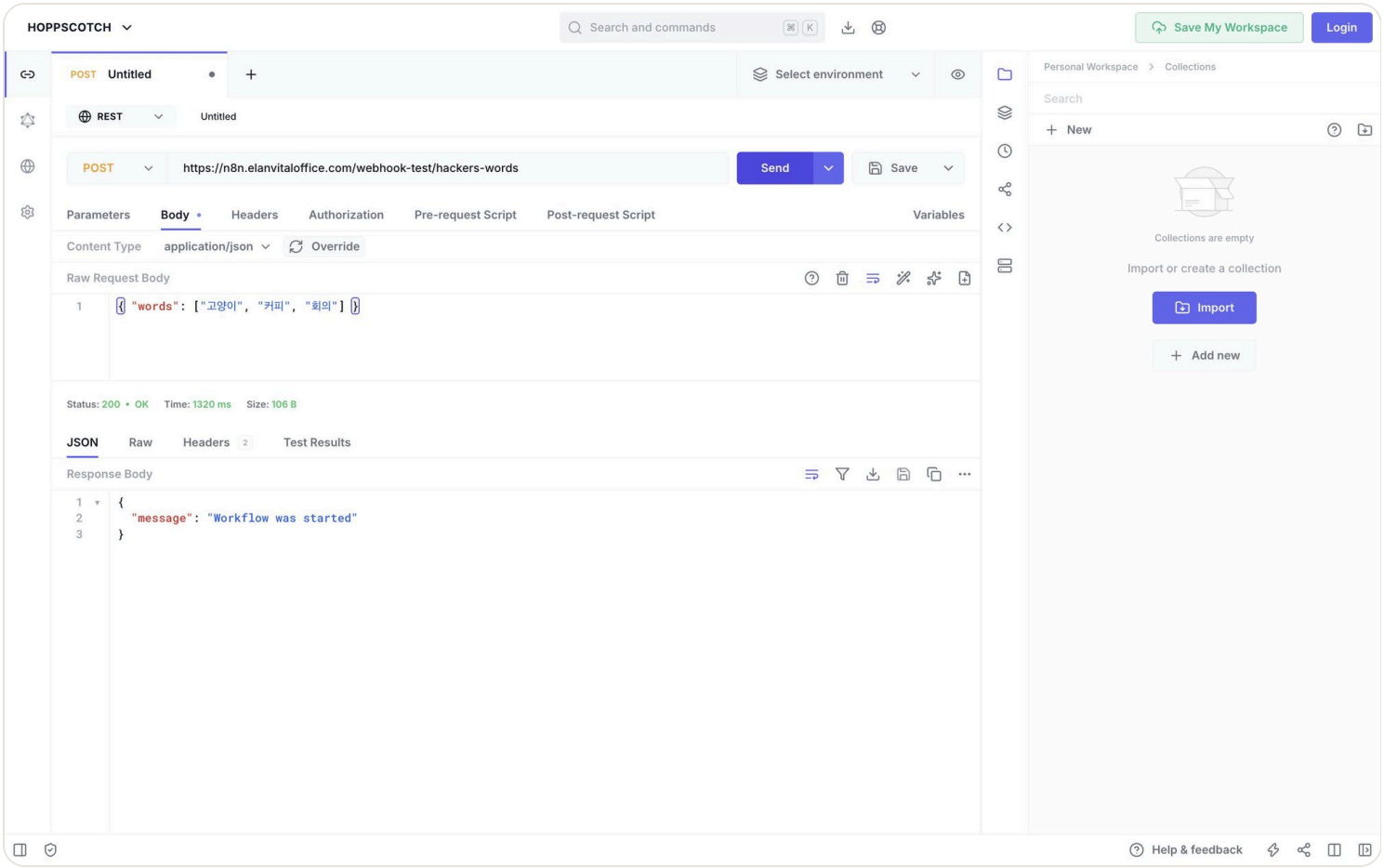
- 택배가 왔는지 **1분마다 문을 열어보는 대신**(= 폴링)
- 택배기사가 **초인종을 누르면 그때 반응**합니다 (= 웹훅)
- **n8n Webhook 노드 = n8n에 달린 초인종 + 우편함 주소**
    - 누군가 이 주소로 **JSON 주문서를 POST** 하면 → 워크플로우가 **자동 시작**

테스트 URL vs 프로덕션 URL = 리허설 vs 본공연

	🤹 **Test URL** `/webhook-test/…`	🎬 **Production URL** `/webhook/…`
언제	만들면서 확인할 때	완성 후 상시 운영
조건	**Listen for test event** 누른 뒤 **딱 1번**	워크플로우 **Publish(활성화)** 후 항상
결과	편집 화면 OUTPUT 에 바로 표시	Executions 탭에서 확인

Hoppscotch — 주문서를 보내는 우체통 📨

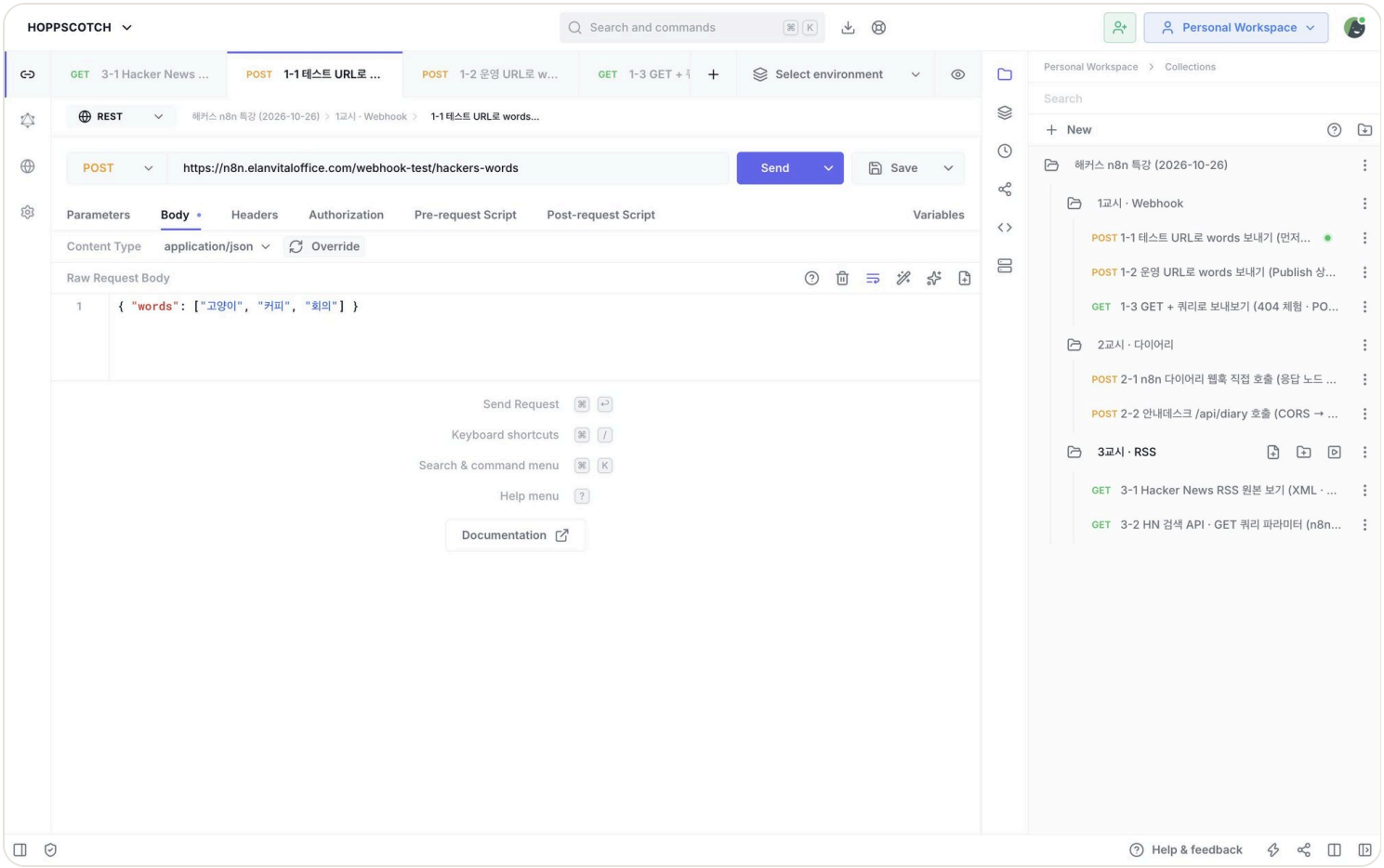
- 웹훅(초인종)을 만들었으면 **누군가 눌러 줘야** 합니다. 실습에서는 그 "누군가"를 우리가 직접 합니다.
- Hoppscotch = **설치 없이 브라우저에서** API 요청을 만들어 보내는 무료 도구 (Postman 의 웹 버전이라고 생각하세요)
- 화면 하나 = **주문서 한 장**: ① 방식(POST) ② 주소(URL) ③ 형식(Content-Type) ④ 내용(Body · JSON) → **Send** → 아래에 **영수증(상태코드)과 답장(응답)**



Hoppscotch 로 n8n 테스트 웹훅에 words 를 보낸 화면 — Status 200 OK

워크스페이스 · 컬렉션 — 주문서 서랍장 📦

- 구글 계정 등으로 **로그인**하면 내 요청이 **Personal Workspace(개인 작업 공간)**에 저장되어 어느 PC 에서나 다시 꺼내 쓸 수 있습니다.
- **컬렉션** = 요청들을 담는 **서랍**, **폴더** = 서랍 속 칸막이. 오늘 강의는 `1교시 · Webhook` / `2교시 · 다이어리` / `3교시 · RSS` 폴더로 정리해 두었습니다.
- 컬렉션은 **Export(내보내기) → 파일 1개** 로 저장하고, 다른 사람은 **Import(가져오기)** 로 그대로 받아 쓸 수 있습니다 → 셀프 학습 키트에 넣어 드립니다 🎁



오늘 강의용 컬렉션 — 교시별 폴더 7개 시나리오

> 💻 Cursor / VS Code 를 쓰는 분은 확장 **REST Client** 의 `.http` 파일로도 똑같이 보낼 수 있습니다. 터미널 `curl`, Postman, 웹페이지의 버튼(2교시!)도 모두 같은 일을 합니다.

📦 셀프 학습 키트 · 보내는 도구

- **hackers-n8n-selfstudy.json** — 오늘 시나리오 7개 Hoppscotch 컬렉션 (Import 용) · **가져오는 방법**
- **webhook.http** (Cursor REST Client) · **curl.sh** (Mac/Linux) · **curl.ps1** (Windows)

실습 1 · 초인종 만들고 눌러보기

